I. Introduction

One of the 5 human senses is hearing. Just as a human can hear and thus differentiate between sounds; a computer if trained should be able to do the same. For a computer to 'read' sounds it needs to be converted into a format it can understand. This comes in 2 forms, waveforms and spectrograms. Waveforms contain information about the amplitudes and when these amplitudes occur. Spectrograms contain information about frequencies, amplitude and time. Waveforms are usually plotted with time on the horizontal axis and amplitude on the vertical axis. Spectrograms are shown using heatmaps where they have time on the horizontal axis, frequency on the vertical axis and a color scale showing the amplitude. The code's purpose is to take a spectrogram/waveform dataset of already classified sounds and produce a machine learning model that can correctly categorise the sounds and thus analyze its ability to correctly categorise unseen data.

II. Method

The dataset used is in the format of a Pandas data frame containing both waveform, spectrograms and their respective labels. The Pandas dataset contains 50 categories and 40 waveforms, spectrograms per category, meaning the dataset is balanced.

First is the spectrogram case. Considering the spectrograms can be considered as images with color, a model that's uses CNNs would be appropriate. As there are 2000 images, using TensorFlow's tf.data.Dataset is particularly useful , as it allows for optimization techniques. First the code obtains the spectrograms and their labels, then it converts (one hot encodes) the labels, then it gets converted into a tf.dataset object (image,label), where the dataset is shuffled and then put into batches of sizes 32. This dataset is then split into training, validation and testing dataset in a 60%,10%,30% split respectively. 3 optimization methods are applied to the dataset, these being, prefetch, autotune, cache. Prefetch obtains the next batch of data whilst training current batch, cache stops repeating unnecessary tasks, autotune makes the amount of data being prefetched change automatically depending on how the training is going. Next is the model, the structure for the model is 3 convolutional neural network layers (each followed with a max pooling layer) with the last one having a tanh activation function due the large amount of negative numbers in the spectrogram data. Then a batch normalization layer to prevent the calculated weights from being too large and thus stopping the 'exploding gradient' problem. This is followed by 2 dense layers with the last one having a SoftMax activation to predictions as probabilities.

```
model = models.Sequential([
    layers.Conv1D(64 , kernel_size = 7, activation = 'relu', input_shape
    layers.MaxPooling1D(pool_size = 3,strides = 2),
    layers.BatchNormalization(),###normalize
    layers.Conv1D(64 , kernel_size = 7, activation = 'relu', input_shape
    layers.MaxPooling1D(pool_size = 3,strides = 2),
    layers.Conv1D(32, kernel_size = 3, strides = 2, activation = 'tanh',
    layers.MaxPooling1D(pool_size = 3,strides = 2),
    layers.BatchNormalization(),



    layers.Flatten(),
    layers.Dense(120, activation = 'tanh'),
    layers.Dropout(0.01),
    layers.Dense(64,activation = 'relu'),
    layers.Dense(50, activation = 'softmax'), ### To give predictions as
])

####using tanh for activation function as some images contain a lot of d
```

Fig1. Convolutional neural network model structure for spectrogram case

An adam optimizer is used with an intial learning rate of 0.001 where tf.optimizer is used as it adapts the learning rate. 50 epochs are used to fit the model to training data and it obtains a training accuracy of 90% and a test accuracy of 81%.

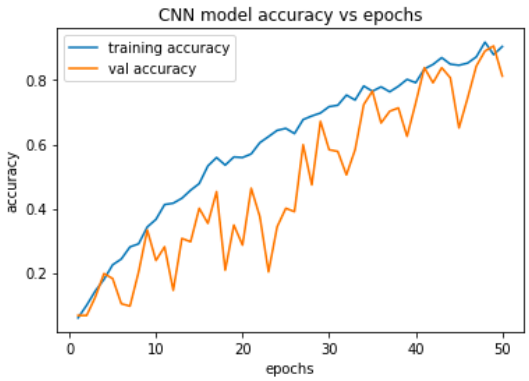


Fig 2. Plot of spectrogram model's train and validate accuracy vs epochs.

Next there is the waveform data. This dataset has a problem, where it is 220500 amplitudes per waveform. 220500 amplitudes is too large of a data size and provides no useful additional information compared to a smaller amount of amplitudes. To confront this problem the code only takes every 50th element in each of the 220500-size array. Similar to the spectrogram case we use the same optimization techniques and train, validation, test sizes. As for the model it is 2 convolutional layers with the first one having 12 features to look for, due to the images having a small amount of patterns. Then there are 2 dense layers again with SoftMax in the last one to give predictions as probabilities.

```
model_2 = models.Sequential([


    layers.Conv1D(12 , 3, activation = 'relu', input_shape = input_model_shape,padding = 'same
    layers.Conv1D(3, 3 , activation = 'relu', input_shape = input_model_shape,padding='same'),
    layers.MaxPooling1D(pool_size = 1, strides =1 ,padding='same'),
    layers.BatchNormalization(),



    layers.Flatten(),
    layers.Dropout(0.1),
    layers.Dense(120, activation = 'relu'),
    layers.Dropout(0.4),
    layers.Dense(64,activation = 'relu'),
    layers.Dense(50, activation = 'softmax'),
])
```

Fig 3. Waveform convolutional network structure

The same model compiling, and optimizer used in the spectrogram case is used in the waveform model. This model has a 93% training accuracy and a 98% testing accuracy.

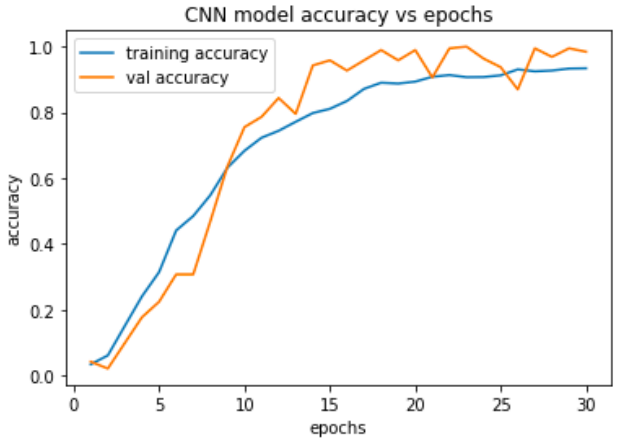


Fig 4. Plot of waveform model's train and validate accuracy vs epochs.

III. Analysis

For both models a confusion matrix was made however, due there being 50 categories a 50 x 50 diagram is hard to understand (can't easily see the exact amount of correct and incorrect predictions due to image quality) so, the precision, recall, F1 scores were also printed. In the case of the spectrogram model the main categories that caused concern were the sea waves, keyboard typing, crickets and laughing sounds.

Category	Precision	Recall	F1 Score
Crickets	0.54	0.92	0.68
Keyboard	0.74	0.54	0.62
Laughing	0.86	1.00	0.93
Sea Waves	0.96	0.82	0.89

Table 1. Spectrogram's model worst performing categories and their respective predict metrics

.

From table 1 and the confusion matrix it shows that the model predicts crickets instead of keyboards and sea waves often. A possible cause would be if the spectrogram images are visually similar.

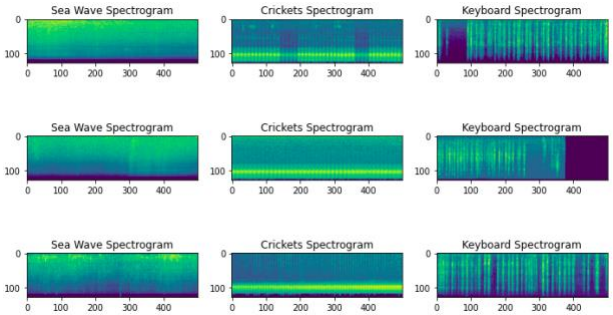


Fig 5. Spectrogram plots for the categories that getting mixed up by the model, to see if the images have any similarity.

From the spectrogram images shown in Fig 5, one could argue that in all the sea wave and cricket plots, there is a colour gradient moving vertically (across the whole image) and horizontally there is a fixed colour (for a given vertical position). However, this visual observation has contradictions as, the direction of the colour gradient is opposite for crickets and sea waves. Furthermore, the keyboard images don't have a constant horizontal colour and neither do the cricket images upon a much closer inspection. This wold suggest that the model, when given a spectrogram for one of the three categories, notices the overall colour gradient pattern and thus misdiagnoses the category. Considering the amount convolutional layers and, the nature of CNN's to identify a pattern anywhere in an image, the similar colour gradient problem would be justified.

In the waveform case the test accuracy gives reason to believe there is overfitting. The confusion matrix and prediction metrics show no valuable information as the test accuracy is almost 100%. Initially the model was being on a 90%,10%,10% train, validate, test dataset size. When this happening sometimes the model would confuse keyboard typing, laughing and fire waveforms. Taking the same approach in the spectrogram case of plotting the waveforms showed no common patterns between the different category waveforms despite, an initial observation that the amplitudes peaks being in a similar shape (which was disproved when the fire waveforms were plotted).

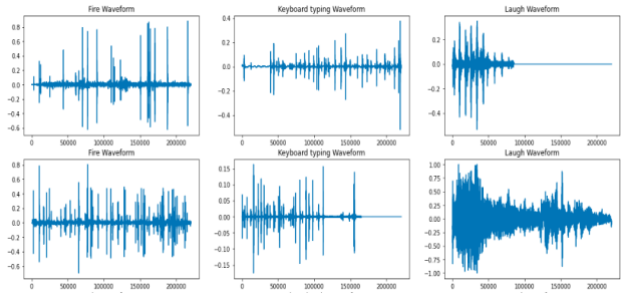


Fig 6. Waveform plots for 3 most commonly confused categories. Plots show no similarity between images.

Based on this evidence, either the model has overfitted or performs very accurately on both seen and unseen data.

IV. Conclusion

Both models give an 80% or higher test accuracy. The spectrogram model incorrectly predicts sea wave, cricket and keyboard sounds most often, possibly due to the similarity of the spectrogram images when looking for a large pattern across the whole image. The waveform models show signs of overfitting but proving this would require more data to be more certain. Thus, to improve this project in the future an image generator should be used to provide more data and better test the models on unseen data which, would also prove whether or not there is overfitting in the waveform model. Another addition would be a feature map to visualise what the model focusing on and to determine whether the colour gradient suggestion is true.